班级选课、成绩系统使用说明

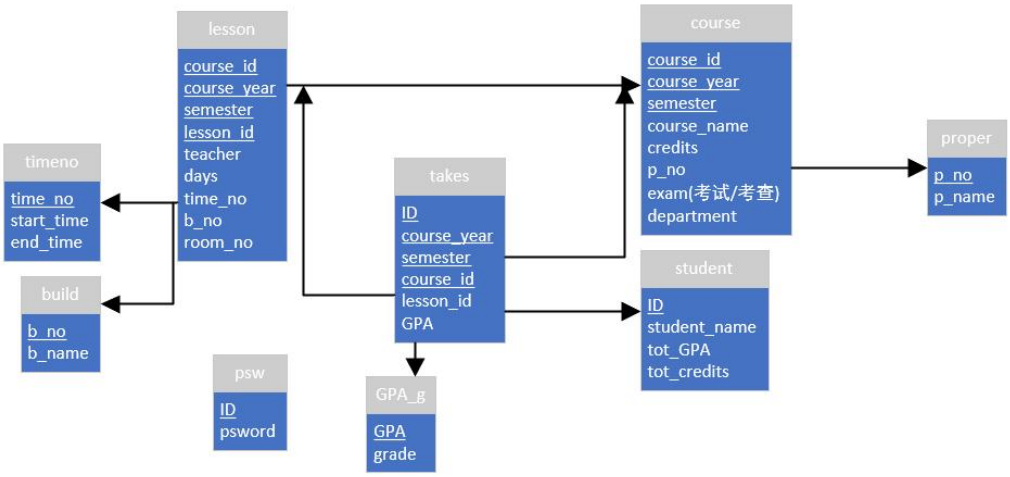
王旌 2019141410308

一、数据库系统简介

本数据库名为力学软件班级选课情况及成绩，由班级管理员（如班主任、班长、班委）进行注册并登录此系统，主要用于添加、删除同学的选课以查看力学软件班全班同学的课表和总学分、成绩，便于管理员在所有人空闲时安排班级活动、查看同学学习情况、方案完成情况和进行成绩帮扶的活动。

二、数据库设计

1）数据库模式图如下：



如图设计表格，在所有表格的外码中，都有保证完整性约束的级联删除更新 on delete cascade、on update cascade；对于 semester，check (semester in ('春','秋'))；对于 exam，check(exam in ('考试','考查'))；对于 days，check(days in('周一','周二','周三','周四','周五','周六','周日'));另外,对于表中所有属性,有必要时应设置默认值或 not null

对于 build、timeno、proper 三个表是固定的，不能改变，主要目的是减少管理员输入时的工作量，只需要用编号代替教学楼、时段、课程属性

```
insert into build values('1','综合楼');
insert into build values('2','一教A');
insert into build values('3','一教B');
insert into build values('4','一教C');
insert into build values('5','一教D');
insert into build values('6','建环楼');
insert into build values('7','体育场');
insert into build values('8','二基楼');
insert into build values('9','文科楼');
```

```
insert into timeno values('1','8:15','9:55');
insert into timeno values('2','8:15','11:00');
insert into timeno values('3','8:15','11:55');
insert into timeno values('4','10:15','11:55');
insert into timeno values('5','13:50','15:30');
insert into timeno values('6','13:50','16:25');
insert into timeno values('7','16:45','18:25');
insert into timeno values('8','19:20','21:00');
insert into timeno values('9','19:20','21:55');
```

```
insert into proper values('1','必修');
insert into proper values('2','选修');
insert into proper values('3','任选');
```

2) 所有功能及对应源码:

输入学生信息:insert into student(ID,student_name) values (%s, %s)

显示学生信息:select ID,student_name from student

删除学生信息:delete from student where ID=%s

插入课程:insert into course values (%s, %s, %s, %s, %s, %s, %s, %s);
insert into lesson values (%s, %s, %s, %s, %s, %s, %s, %s, %s)

显示课程列表:
select
course_id,course_name,lesson_id,course_year,semester,teacher,credits,p_name,exam,department,b_name,room_no,days,start_time,end_time from lesson natural join course

natural join timeno natural join proper natural join build

删除课程:delete from course where (course_id,semester,course_year)=(%s, %s, %s)

增加一位同学的选课:
insert into takes(ID, course_year,semester,course_id,lesson_id,GPA) values(%s,%s,%s,%s,%s,%s)

删除一位同学的选课:
delete from takes where(ID, course_year,semester,course_id) =(%s,%s,%s,%s)

显示一位同学在某学期的课表:
select course_id,course_name,lesson_id,teacher,credits,p_name,exam,department,b_name,room_no,days,start_time,end_time from takes natural join course natural join lesson natural join timeno natural join proper natural join build where (ID,course_year,semester)=(%s,%s,%s)

显示一位同学的总学分和 GPA:select student_name,tot_GPA,tot_credits from student where ID=%s

3) 添加 trigger:为了实现显示一位同学的总学分和 GPA,在对学生选课表更新(添加,删除)时,需自动累加所有课程的学分以及根据课程学分和 GPA 计算平均(总)GPA

源码如下:

```
 DELIMITER $
    create trigger totalcredits1 after insert on takes
    for each row
    begin
    update student
    set tot_credits=tot_credits+(select credits from course where (new.course_id,new.course_year,new.semester)=(course.course_id,course.course_year,course.semester))
    where new.GPA>0 and ID=new.ID;/*自动将 char 转换为数值*/
    end $
    DELIMITER ;

     DELIMITER $
    create trigger totalcredits2 after update on takes
    for each row
    begin
    update student
    set tot_credits=tot_credits+(select credits from course where
```

```
(new.course_id,new.course_year,new.semester)=(course.course_id,course.course_year,
course.semester))
    where new.GPA>0 and ID=new.ID;
    end $
    DELIMITER ;

  DELIMITER $
    create trigger totalcredits3 after delete on takes
    for each row
    begin
    update student
    set tot_credits=tot_credits-(select credits from course where
(old.course_id,old.course_year,old.semester)=(course.course_id,course.course_year,cou
rse.semester))
    where old.GPA>0 and ID=old.ID;
    end $
    DELIMITER ;

    drop view if exists totGPA;

    create view totGPA(ID,t,credits) as
    select ID,takes.GPA*course.credits,credits from takes natural join course;/*自动将
char 转换为数值*/

     DELIMITER $
    create trigger totalGPA1 after insert on takes
    for each row
    begin
    update student
    set tot_GPA=(select sum(distinct t) from totGPA where ID=new.ID)/(select
sum(distinct credits) from totGPA where ID=new.ID)
    where new.GPA>0 and ID=new.ID;
    end $
    DELIMITER ;

   DELIMITER $
    create trigger totalGPA2 after update on takes
    for each row
    begin
    update student
    set tot_GPA=(select sum(distinct t) from totGPA where ID=new.ID)/(select
sum(distinct credits) from totGPA where ID=new.ID)
    where new.GPA>0 and ID=new.ID;
    end $
```

```
DELIMITER ;

 DELIMITER $
create trigger totalGPA3 after delete on takes
for each row
begin
update student
set tot_GPA=(select sum(distinct t) from totGPA where ID=old.ID)/(select sum(distinct credits) from totGPA where ID=old.ID)
where old.GPA>0 and ID=old.ID;
end $
DELIMITER ;
```

4) 用户界面:

为了数据库的安全性,数据库的源码以及表结构是不能给用户看到的,因此有必要制作gui,使用系统必须注册/登录,并且在每个功能中只能让用户输入必要的项且需遵照输入格式,界面还应有输入合法性检测,若输入非法(格式错误、需非空、违反参照性约束)需跳出提示框,并让用户重新输入,并事务回退以不对数据库有所影响;若使用功能添加、删除成功也应跳出成功、失败提示框

本数据库使用 python easygui 进行用户界面的编写,运用 multenterbox、msgbox、buttonbox、multpasswordbox 以及 prettytable 库进行界面的实现,导入 pymysql 库,运用如下代码连接到具体数据库,实现在用户界面对数据库进行操作:

```
conn=pymysql.Connect(

    host='localhost',

    port=3306,

    user='root',

    passwd=?,

    db=?,

    charset='utf8'

    )
   cursor=conn.cursor()
```

对数据库操作进行防错检测:(以插入为例)

```
try:
        affected_rows = cursor.execute('insert into psw values
```

```
(%s, %s)',(s[0],s[1]) )
                if s[0]=='' or s[1]=='':
                        t.msgbox(msg='输入非空！ ', title='fail', ok_button='ok')
                if affected_rows == 1:
                        t.msgbox(msg='注册成功', title='success', ok_button='ok')
                        conn.commit()
    except pymysql.MySQLError:
                        conn.rollback()#数据库操作出错时进行事务回滚
                        t.msgbox(msg='注册失败', title='fail',
ok_button='ok')#msgbox 用于跳出正确、错误提示
```

对数据库操作进行防错检测:(以删除为例)

```
    try:
        affected_rows = cursor.execute('delete from course where
(course_id,semester,course_year)=(%s, %s, %s)',(s[0],s[2],s[1]) )
                if affected_rows == 1:
                        conn.commit()
                        t.msgbox(msg='删除成功！', title='success',
ok_button='ok')
        except pymysql.MySQLError:
                        conn.rollback()#数据库操作出错时进行事务回滚
                        t.msgbox(msg='删除失败', title='fail', ok_button='ok')
```

对于显示功能，将查询结果的每一行添加到 PrettyTable 的每一行并显示给用户：

```
    cursor.execute('select student_name,tot_GPA,tot_credits from student where
ID=%s',s[0])
                        rr=cursor.fetchall()
                        tb = pt.PrettyTable( ["姓名", "平均 GPA","总学分"])
                        for row in rr:
                         tb.add_row([row[0],row[1],row[2]] )
```

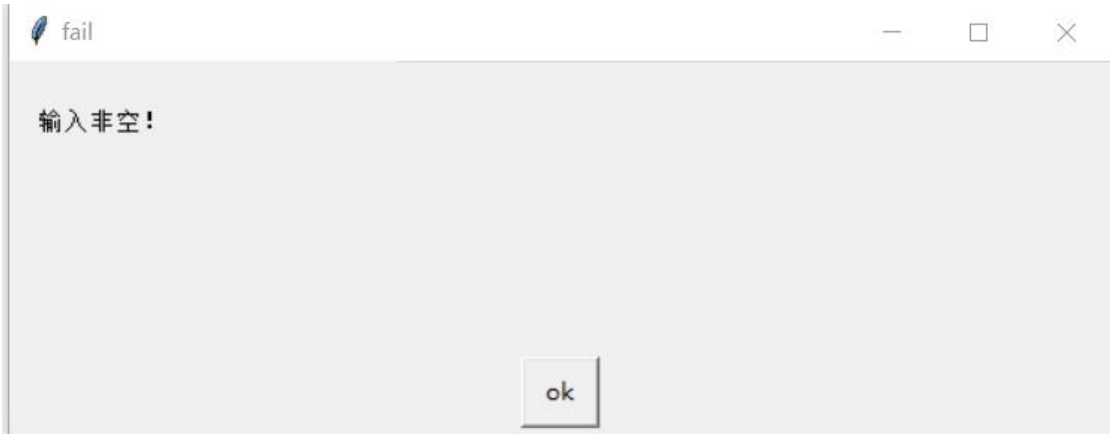
三、数据库系统使用方法及功能正确性测试

在这里,初始数据库为空，用用户界面插入、删除数据进行功能正确性的验证:

mysql 代码第一句在不同电脑上需做更改,应用于具体的数据库名:

```
use ddl_mysql;
```

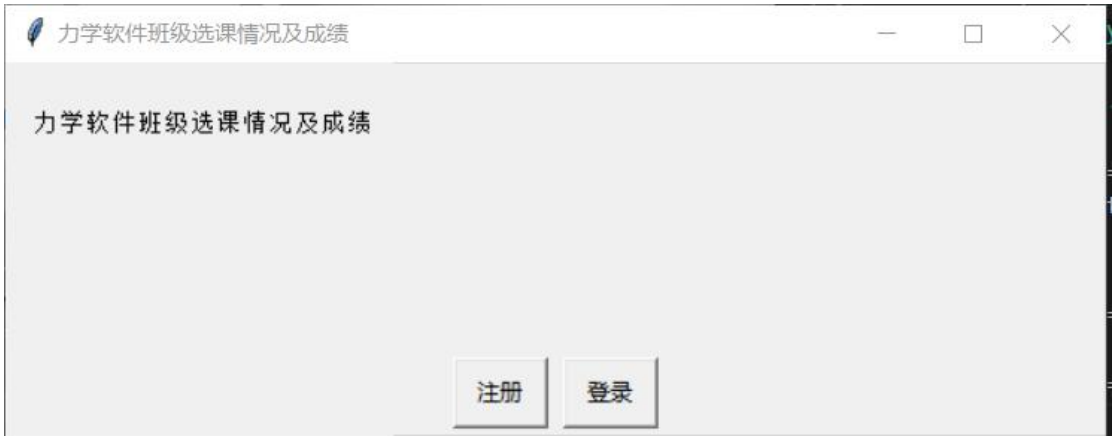
所有输入界面都有防错检测，若输入要求非空，不按要求输入点 ok 则会提示：



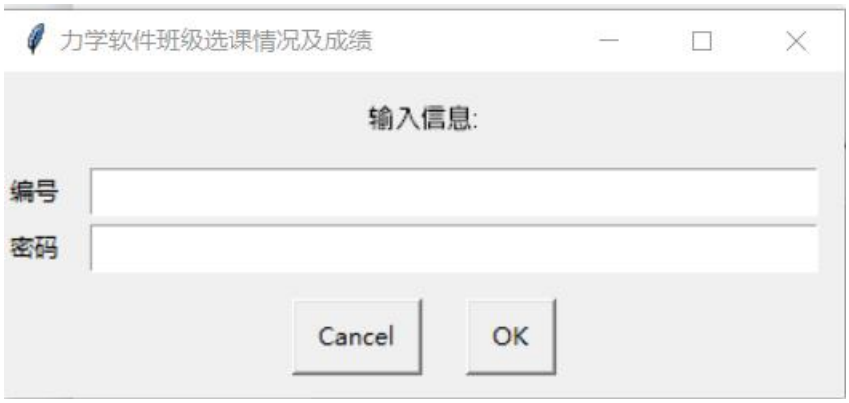
具体使用流程:

1、连接到具体数据库，并输入密码:

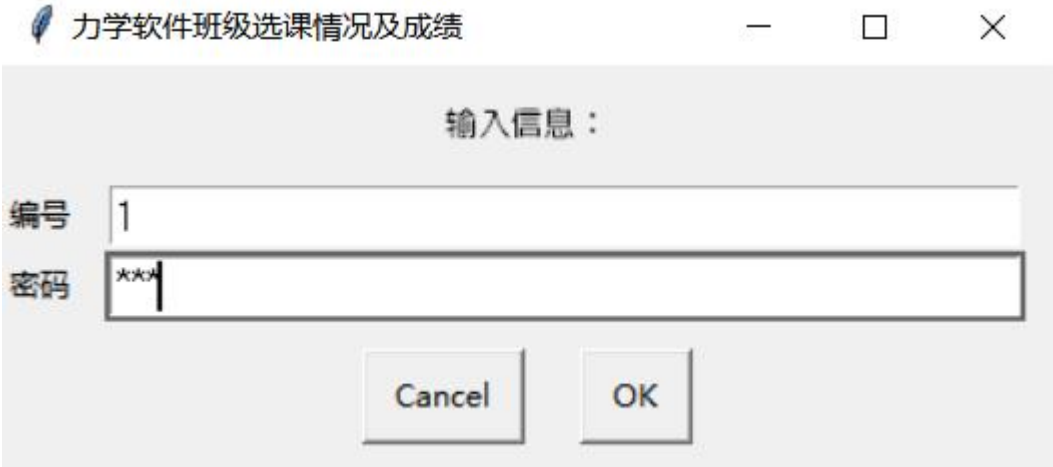
2、进行注册或登录:

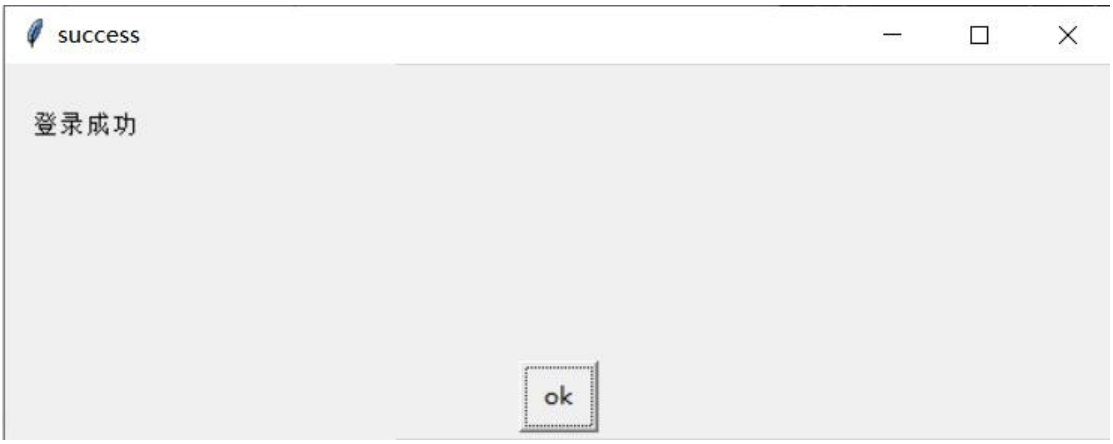


注册:

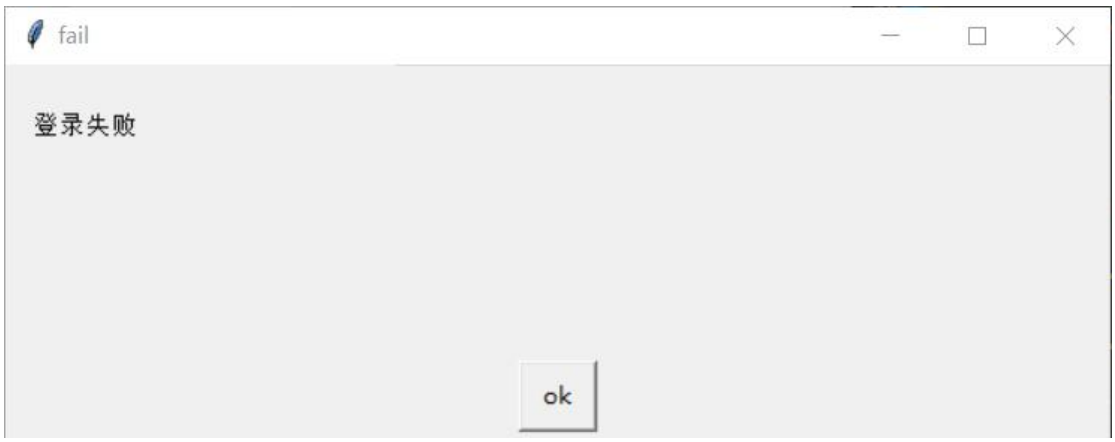


登录:



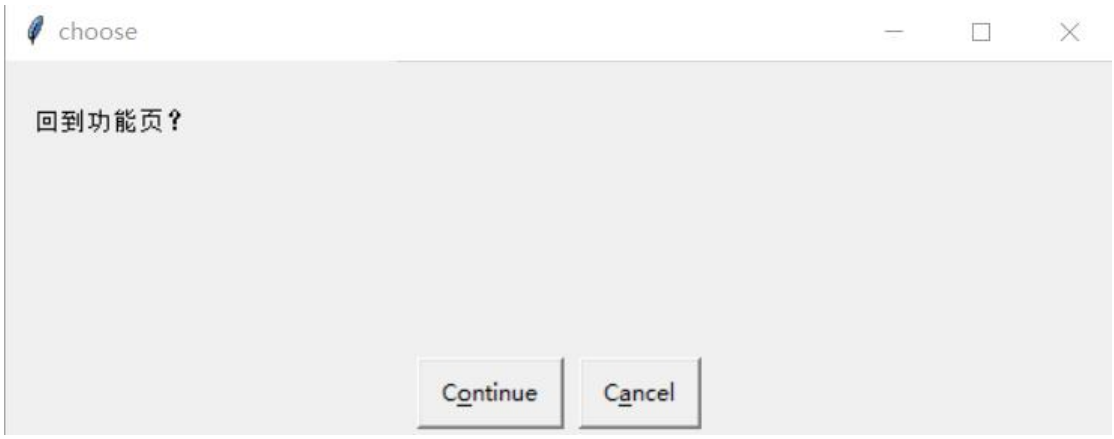


注册和登录均有成功和失败的提示，如:

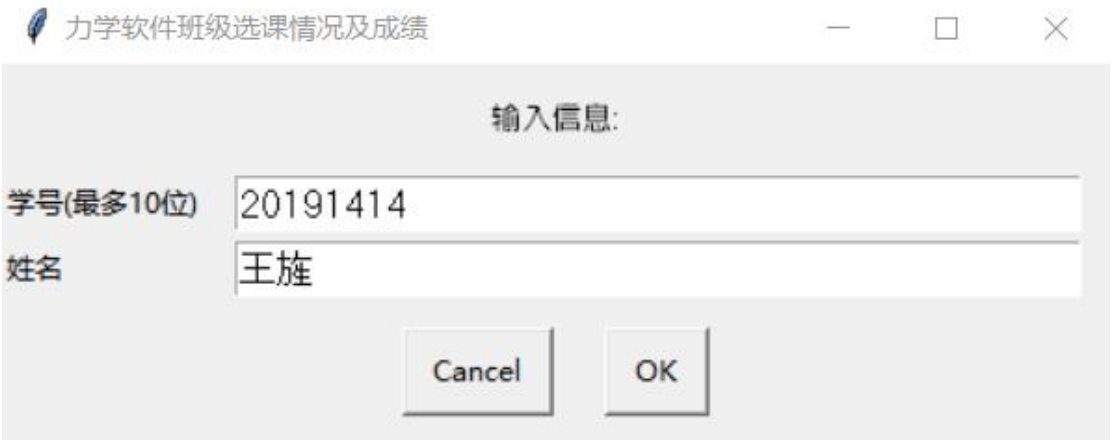


3、进入功能菜单，具有功能：输入学生信息、显示学生信息、删除学生信息、插入课程、显示课程列表、删除课程、增加一位同学的选课、删除一位同学的选课、显示一位同学在某学期的课表、显示一位同学的总学分和 GPA，并可退出系统：

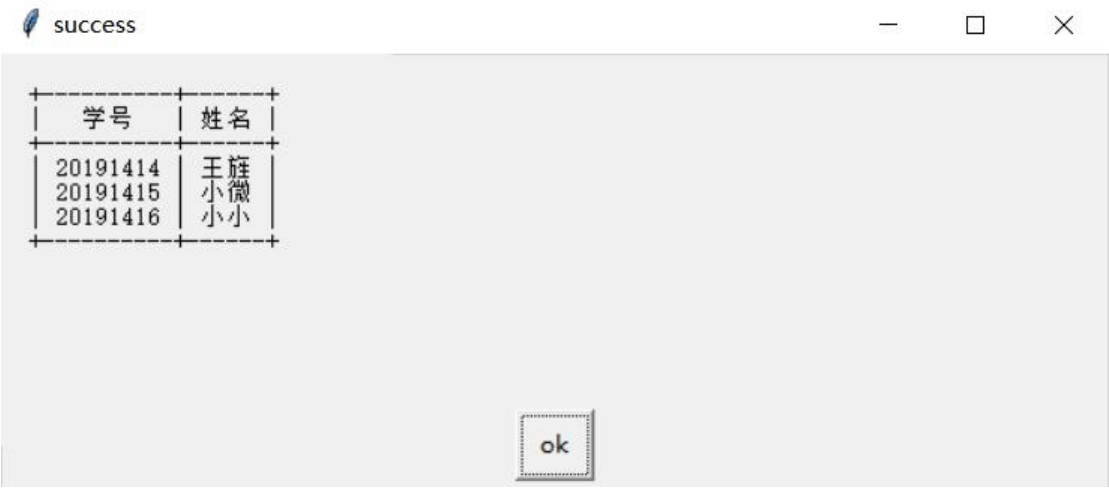
在所有功能中，输入完成点 ok 有输入/删除成功/失败的提示框，输入完成后，会提示是否回到功能页继续使用系统：



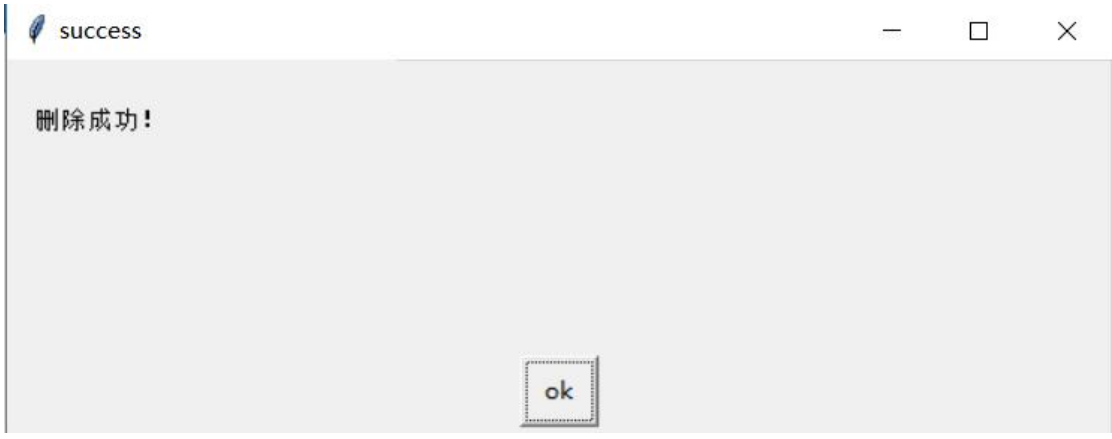
1）点输入学生信息，插入三条：

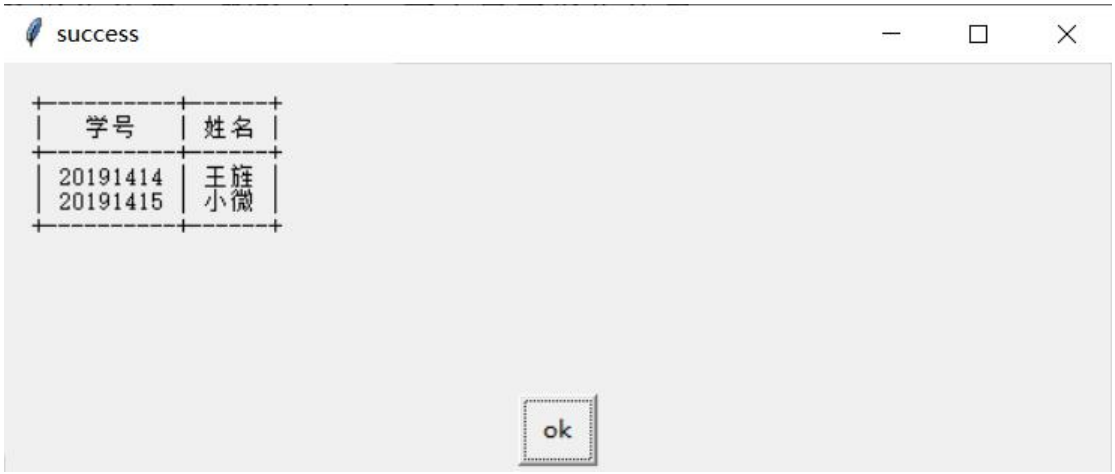


2）点显示学生信息：（说明 1）成功）

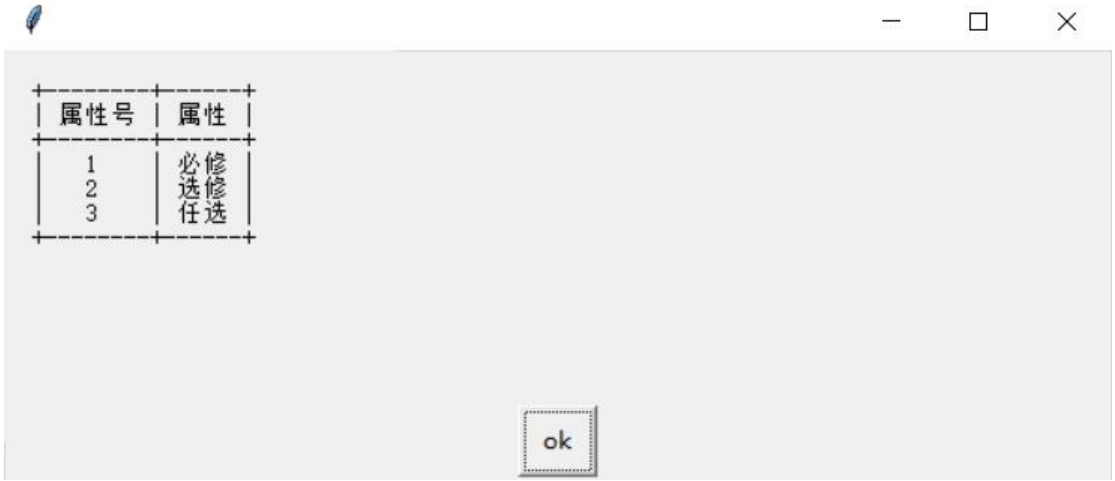


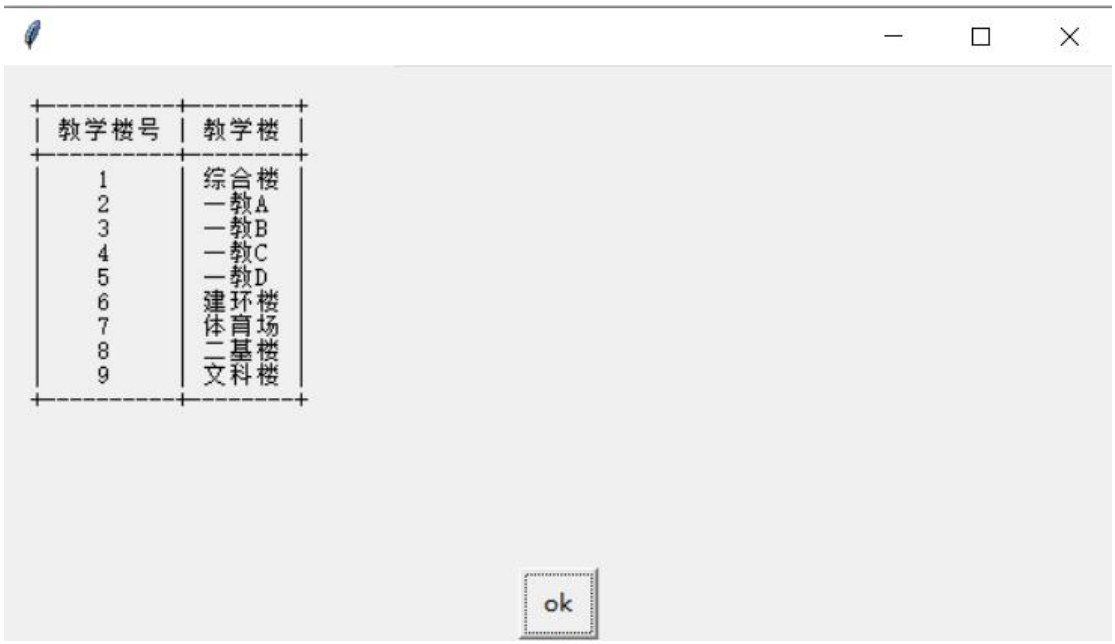
3）点删除学生信息，删除小小，再点显示学生信息：

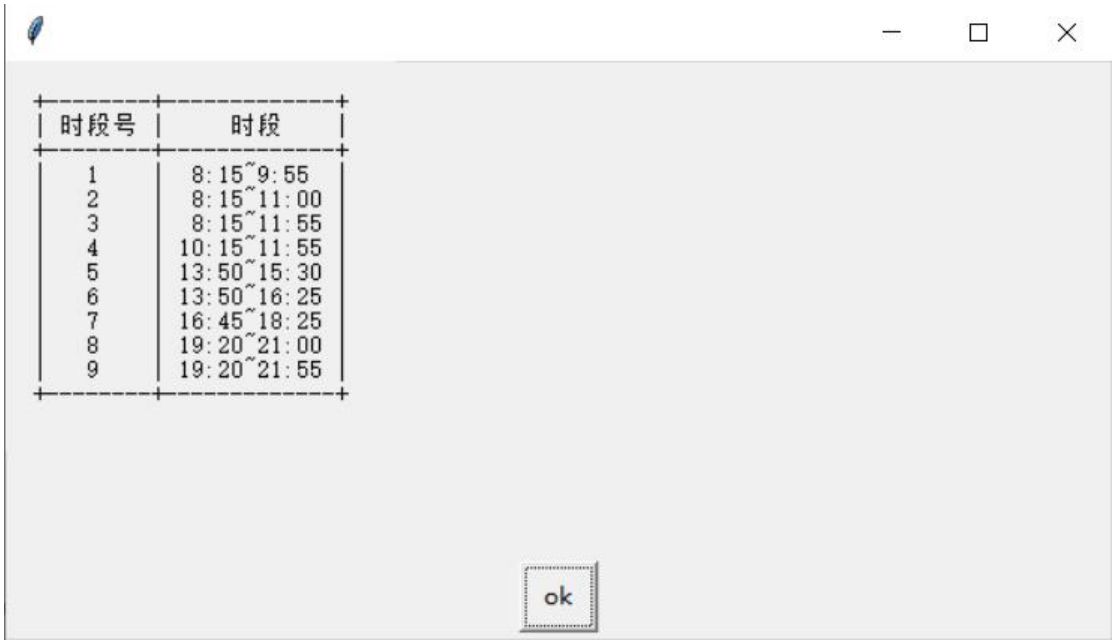




4）点插入课程，首先出现三个提示框，并插入两门课：

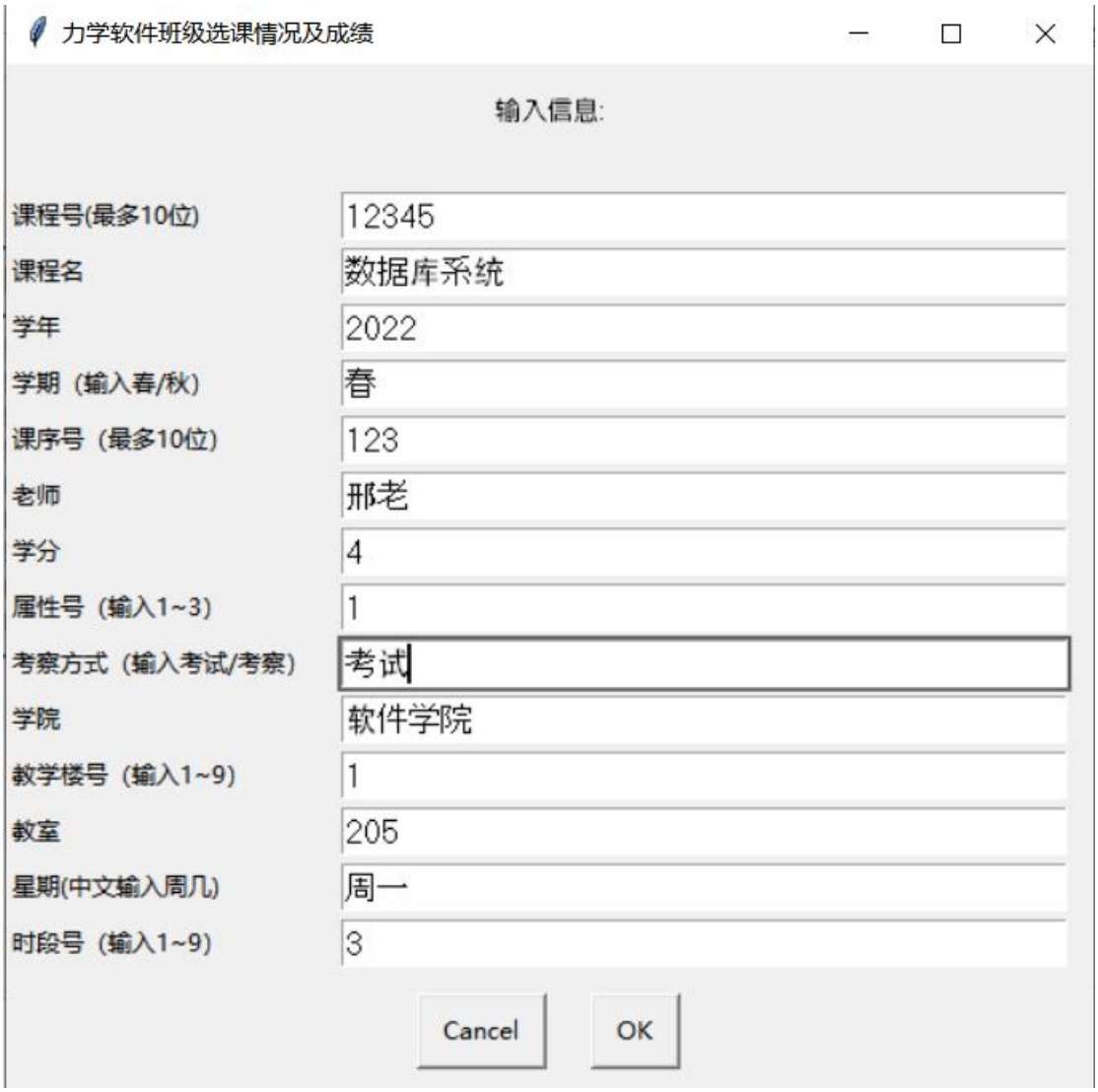


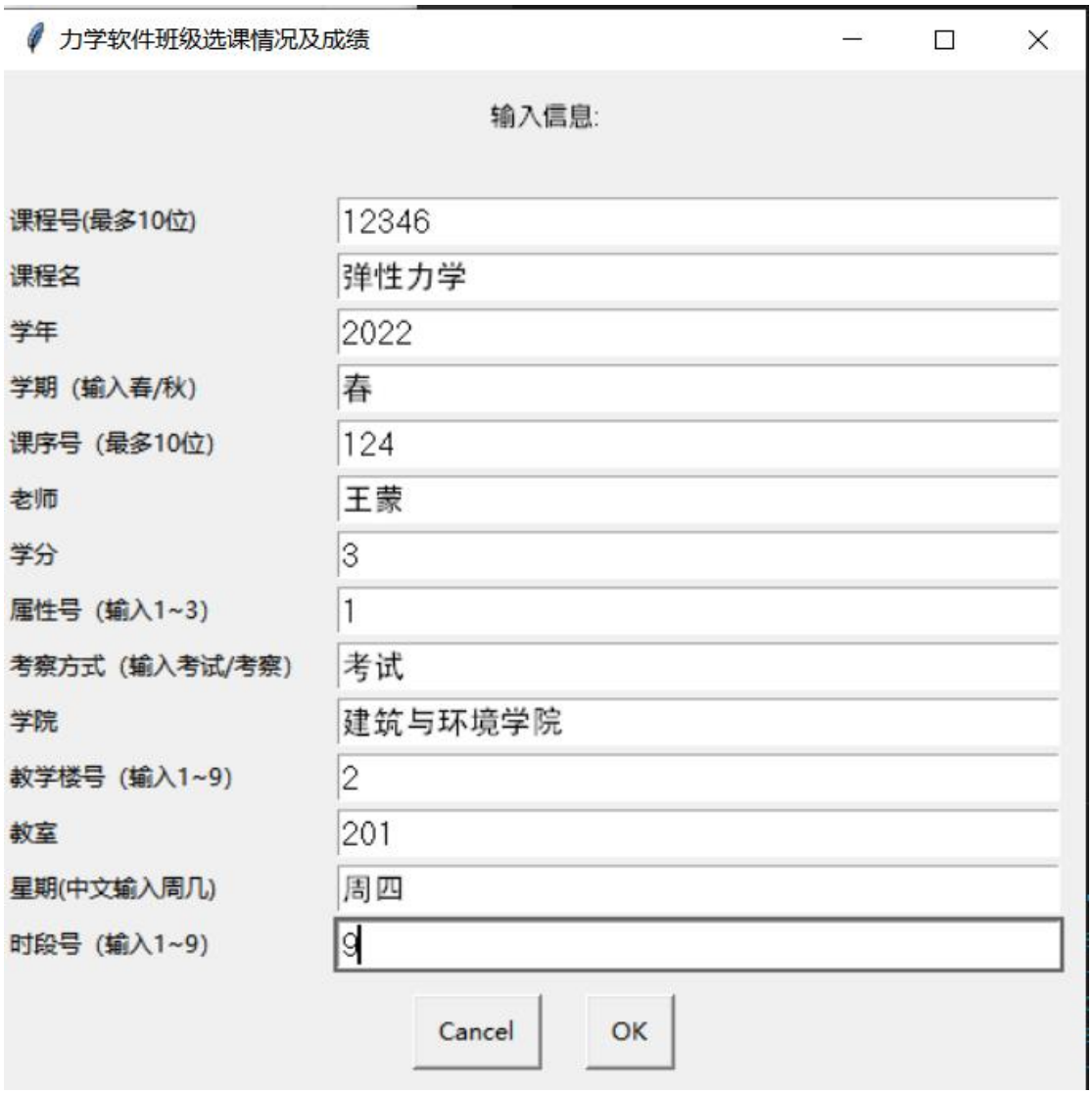




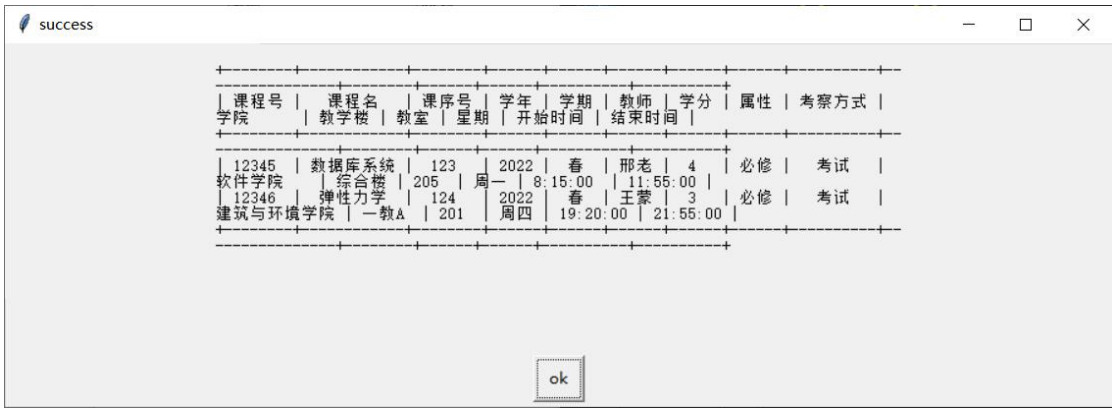
时段号	时段
1	8:15~9:55
2	8:15~11:00
3	8:15~11:55
4	10:15~11:55
5	13:50~15:30
6	13:50~16:25
7	16:45~18:25
8	19:20~21:00
9	19:20~21:55

ok

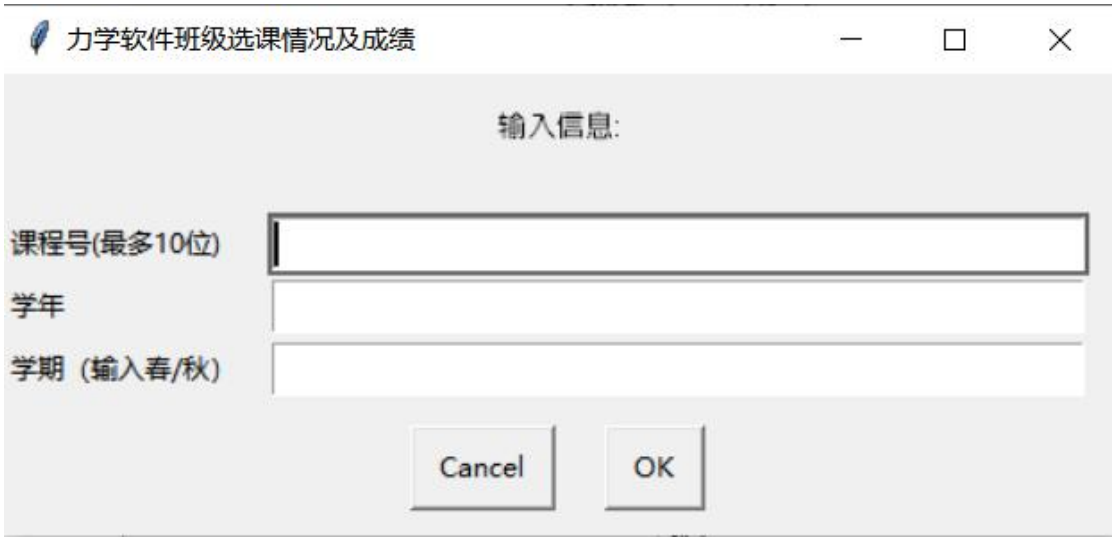




5）点显示课程列表：（说明 4）添加成功）

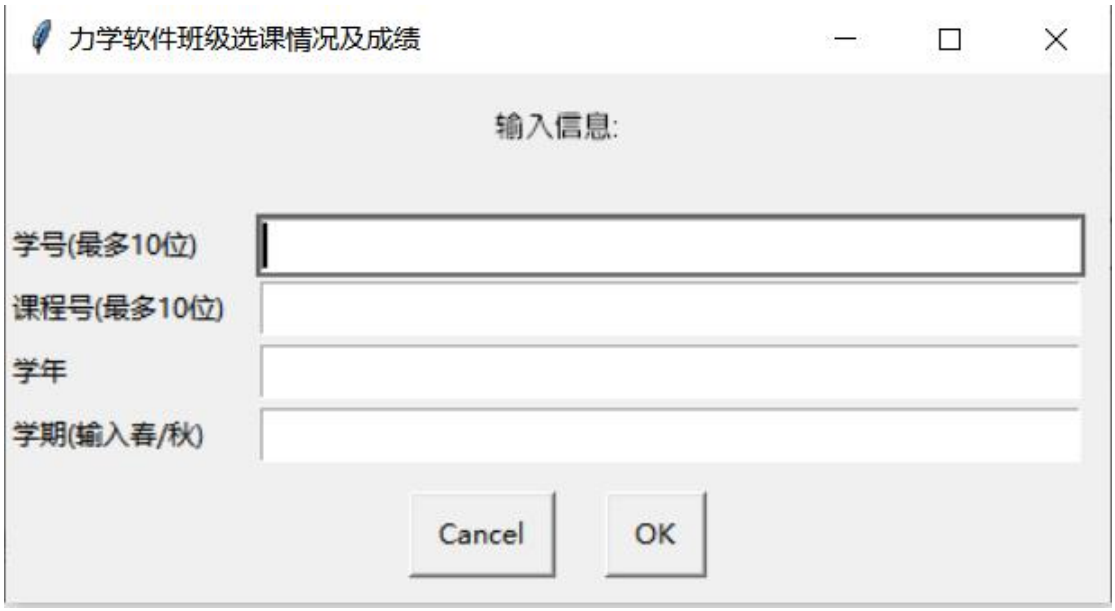


6）删除课程同理

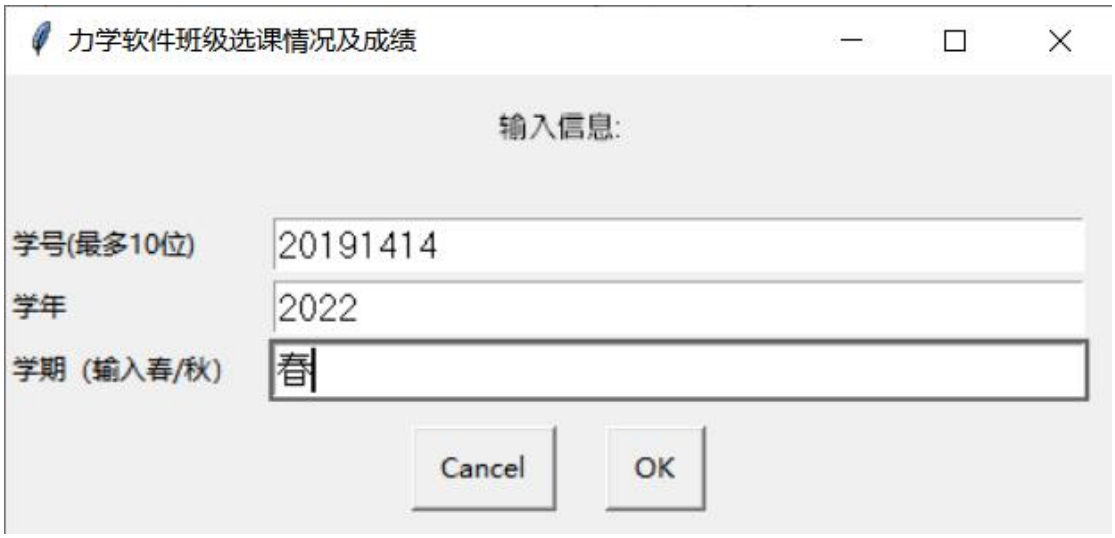


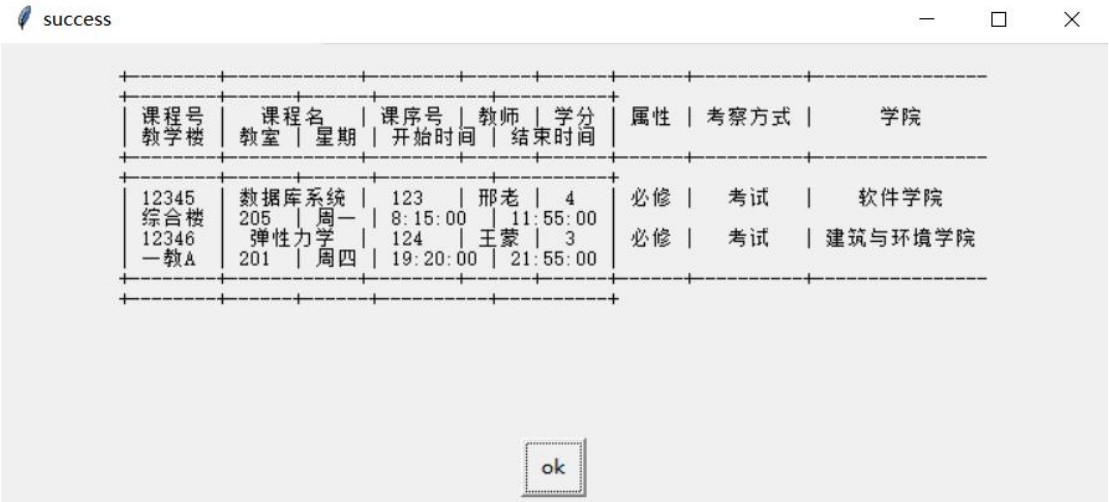
7）点增加一位同学的选课，添加王旌的两门选课:

8）删除选课同理

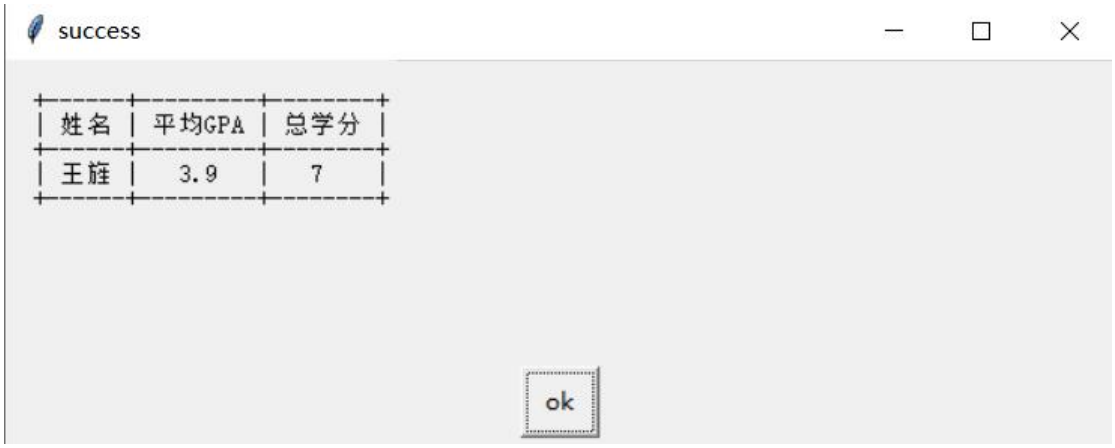


9）点显示一位同学在某学期的课表，这里显示王旌的课表：（说明 7）成功）





10）点显示一位同学的总学分和 GPA，这里显示王旌的：（说明以上步骤皆成功）


success
姓名 | 平均GPA | 总学分
王旌 | 3.9 | 7
ok